

Parameter safety means validating and protecting the values passed to a function before they are used, ensuring the function behaves correctly and securely. It prevents crashes, undefined behavior, incorrect results, and security issues.

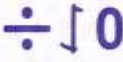
Major instances where parameter safety is required in C.

Parameter Type	Safety Check	Example
Integer	Check valid range	Age ≥ 0 , Marks 0–100
Floating-point	Check valid range	Temperature $\geq -273.15^{\circ}\text{C}$
Division	Denominator $\neq 0$	a / b
Array	Size > 0	Array processing
Array Index	$0 \leq \text{index} < \text{size}$	Accessing arr[index]
Matrix	Compatible dimensions	Matrix multiplication
Pointer	Not NULL	if(ptr == NULL)
Dynamic Memory	Allocation successful	malloc() return value
String	Not NULL and within size	Prevent buffer overflow
Character	Valid character	Menu choice (A–Z, 1–5)
Structure Pointer	Not NULL	Accessing structure members
File Pointer	File opened successfully	fopen()
Function Pointer	Not NULL	Callback functions
Recursion	Valid base case	Prevent infinite recursion
Loop Count	Positive limit	n > 0
Menu Choice	Valid option	1–6 in switch
Search Key	Valid input	Searching an array
Sorting	Non-empty array	Sorting algorithms
Linked List	Head pointer valid	Traversing a list
Queue	Not full / Not empty	Enqueue/Dequeue
Stack	Overflow/Underflow	Push/Pop
Binary Search	Array must be sorted	Binary search algorithm
Mathematical Functions	Domain validation	sqrt(x) where $x \geq 0$
Factorial	Non-negative integer	n ≥ 0
Power	Handle special cases	0^0 , negative exponents
Modulus	Divisor $\neq 0$	a % b
Date	Valid day, month, year	$1 \leq \text{month} \leq 12$
Time	Valid hours, minutes	0–23, 0–59
Salary	Positive value	Payroll processing
Bank Transaction	Positive amount	Deposit/Withdraw
Withdrawal	Balance sufficient	Prevent overdraft
Student Marks	0–100	Result processing
Roll Number	Positive/Unique	Student database
Employee ID	Unique	Employee management
Password	Length/complexity	User authentication
Filename	Valid path	File operations
Input Buffer	Prevent overflow	Safe fgets() usage
Command-line Arguments	Check argc	Program arguments
Enum Values	Within defined range	Enumeration parameters
Boolean Flag	0 or 1	Configuration flags

Common Parameter Safety Techniques in C

Common Parameter Safety Techniques in C

1. **Range Validation:**
if (marks < 0 || marks > 100)
2. **NULL Pointer Check**
if (ptr == NULL)
return;
3. **Array Bounds Check:**
if (index >= 0 && index < size)
4. **Division-by-Zero Check**
if (b == 0)
5. **Memory Allocation Check**
if (ptr == NULL) {
printf("Memory allocation failed");
}
}
6. **File Open Check**
if (fp == NULL)
7. **Input Validation**
if (scanf("%d", &n) != 1)
8. **Use of const for Read-Only Parameters**
void display(const int arr[], int n);
9. **String Length Validation**
if (strlen(name) >= MAX_SIZE)
10. **Menu Choice Validation**
if (choice < 1 || choice > 6)

S.No.	TECHNIQUE	DESCRIPTION	CODE EXAMPLE	PURPOSE
1	RANGE VALIDATION 	Checks if a value lies within the expected range.	<pre>if (marks < 0 marks > 100) printf("Invalid marks");</pre>	Prevents invalid values from causing errors.
2	NULL POINTER CHECK 	Ensures pointer does not point to NULL.	<pre>if (ptr == NULL) return;</pre>	Avoids dereferencing NULL pointers and runtime crashes.
3	ARRAY BOUNDS CHECK 	Checks if an index is within valid array bounds.	<pre>if (index >= 0 && index < size) arr[index] = value;</pre>	Prevents out-of-bounds access and memory corruption.
4	DIVISION-BY-ZERO CHECK 	Ensures divisor is not zero before division.	<pre>if (b == 0) printf("Division by zero!\n"); else result = a / b;</pre>	Avoids runtime errors (SIGFPE).
5	MEMORY ALLOCATION CHECK 	Verifies if memory was allocated successfully.	<pre>ptr = (int*)malloc(sizeof(int)); if (ptr == NULL) { printf("Memory allocation failed"); exit(1); }</pre>	Prevents using unallocated memory which can crash program.
6	FILE OPEN CHECK 	Checks if file was opened successfully.	<pre>fp = fopen("data.txt", "r"); if (fp == NULL) printf("Cannot open file");</pre>	Avoids operations on invalid file pointers.
7	INPUT VALIDATION 	Ensures user input is valid and read successfully.	<pre>if (scanf("%d", &n) != 1) { printf("Invalid input"); return; }</pre>	Prevents incorrect input causing unexpected behavior.
8	USE OF const FOR READ-ONLY PARAMETERS 	Prevents function from modifying input data.	<pre>void display(const int arr[], int n); // arr cannot be modified inside the function</pre>	Improves safety and clarity of function intent.
9	STRING LENGTH VALIDATION 	Checks if string length is within allowed limit.	<pre>if (strlen(name) >= MAX_SIZE) printf("Name too long!");</pre>	Prevents buffer overflow.
10	MENU CHOICE VALIDATION 	Ensures user's menu choice is within valid range.	<pre>if (choice < 1 choice > 6) printf("Invalid choice!");</pre>	Prevents invalid operations in menu-driven programs.

Programming Laboratory Exercise

Experiment1: Menu-Driven Matrix Operations Using Functions and Parameter Safety

Objective: Develop a menu-driven C program to perform various matrix operations using user-defined functions. The program should demonstrate modular programming, parameter passing, and parameter safety by validating all inputs before invoking the corresponding functions.

Problem Statement: Implement a menu-driven program using the switch statement to perform the following matrix operations:

1. Read Matrix
2. Matrix Addition
3. Matrix Subtraction
4. Matrix Multiplication
5. Matrix Transpose
6. Display Matrix
7. Exit

Functions to Implement

- readMatrix()
- addMatrix()
- subtractMatrix()
- multiplyMatrix()
- transposeMatrix()
- displayMatrix()

Parameter Safety Requirements

Before calling the corresponding function, implement the following parameter safety checks in the main() function:

1. Validate that the number of rows and columns is greater than 0.
2. Ensure that the matrix dimensions do not exceed the predefined maximum size (MAX).
3. Validate all user inputs using the return value of scanf().
4. Verify that matrices have been read before performing any operation or displaying them.
5. Before Addition and Subtraction, ensure that both matrices have identical dimensions.
6. Before Multiplication, ensure that the number of columns of the first matrix is equal to the number of rows of the second matrix.
7. Validate that the user's menu choice is within the valid range (1–7).
8. If any validation fails, display an appropriate error message and return to the main menu without terminating the program.

Input and Output – Matrix Operations Program

Case 1: Read Matrices and Display

Input

1. Read Matrices
2. Addition
3. Subtraction
4. Multiplication
5. Transpose (Matrix A)
6. Display Matrices
7. Exit

Enter your choice: 1

Enter rows and columns of Matrix A: 2 2

Enter rows and columns of Matrix B: 2 2

Enter Matrix A:

1 2
3 4

Enter Matrix B:

5 6
7 8

Enter your choice: 6

Output

Matrix A:

1 2
3 4

Matrix B:

5 6
7 8

Case 2: Matrix Addition

Input

Enter your choice: 2

Output

Result of Addition:

6 8
10 12

Case 3: Matrix Subtraction

Input: Enter your choice: 3

Output

Result of Subtraction:

-4 -4
-4 -4

Case 4: Matrix Multiplication

Input: Enter your choice: 4

Output

Result of Multiplication:

19 22

43 50

Case 5: Matrix Transpose

Input: Enter your choice: 5

Output

Transpose of Matrix A:

1 3

2 4

Case 6: Invalid Menu Choice

Input: Enter your choice: 9

Output: Invalid menu choice!

Case 7: Invalid Matrix Dimensions

Input: Enter your choice: 1

Enter rows and columns of Matrix A:

12 3

Output: Invalid dimensions for Matrix A!

Case 8: Addition with Different Dimensions**Input**

Matrix A : 2 2

Matrix B : 3 2

Enter your choice: 2

Output: Addition not possible.

Case 9: Multiplication with Incompatible Dimensions**Input**

Matrix A : 2 3

Matrix B : 2 2

Enter your choice: 4

Output: Multiplication not possible.

Case 10: Operation Before Reading Matrix

Input: Enter your choice: 2

Output: Read matrices first!

Case 11: Exit

Input: Enter your choice: 7

Output: Program Terminated.

Experiment 2: Bank Account Management System Using Parameter Passing and Parameter Safety

Objective: Develop a menu-driven C program using user-defined functions to simulate basic bank account operations. The program should demonstrate parameter passing, parameter safety, modular programming, and the use of return values.

Problem Statement: Implement a menu-driven Bank Account Management System using the switch statement to perform the following operations:

1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit

Functions to Implement

- createAccount()
- deposit()
- withdraw()
- checkBalance()
- displayAccount()

Parameter Passing Requirements

Design the program such that the following parameter passing concepts are demonstrated:

Function	Input Parameters	Return Value
createAccount(int accNo, char name[], float balance)	Account Number, Customer Name, Initial Balance	Initial Balance
deposit(float balance, float amount)	Current Balance, Deposit Amount	Updated Balance
withdraw(float balance, float amount)	Current Balance, Withdrawal Amount	Updated Balance
checkBalance(float balance)	Current Balance	Current Balance
displayAccount(int accNo, char name[], float balance)	Account Number, Customer Name, Current Balance	None (void)

Parameter Safety Requirements: Implement the following safety checks in the main() function before invoking the corresponding function:

1. Validate that the Account Number is a positive integer.
2. Validate that the Customer Name is not empty.
3. Ensure that the Initial Balance is greater than or equal to 0.
4. Validate all numeric inputs using the return value of scanf().
5. Ensure that the Deposit Amount is greater than 0.

6. Ensure that the Withdrawal Amount is greater than 0.
7. Verify that the withdrawal amount does not exceed the available balance.
8. Ensure that an account has been created before performing Deposit, Withdraw, Check Balance, or Display operations.
9. Validate that the user's menu choice is within the valid range (1–6).
10. If any validation fails, display an appropriate error message and return to the main menu without terminating the program.

OR

Parameter Safety Implemented in main()

- ✓ Menu choice validation
- ✓ scanf() input validation
- ✓ Positive account number check
- ✓ Non-empty customer name validation
- ✓ Initial balance ≥ 0
- ✓ Deposit amount > 0
- ✓ Withdrawal amount > 0
- ✓ Withdrawal amount $\leq$ current balance
- ✓ Account creation check before other operations
- ✓ Error messages with return to the main menu

Sample Input and Output

Case 1: Create Account

Input

BANK ACCOUNT MANAGEMENT SYSTEM

1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit

Enter your choice : 1
Enter Account Number : 1001
Enter Customer Name : Rahul
Enter Initial Balance : 5000

Output

Account Number : 1001
Customer Name : Rahul
Account Created Successfully.

Case 2: Deposit Money

Input

Enter your choice : 2

Enter Deposit Amount : 2000

Output

Deposit Successful.

Case 3: Withdraw Money

Input

Enter your choice : 3

Enter Withdrawal Amount : 1500

Output

Withdrawal Successful.

Case 4: Check Balance

Input

Enter your choice : 4

Output

Current Balance : 5500.00

Case 5: Display Account Details

Input

Enter your choice : 5

Output

----- ACCOUNT DETAILS -----

Account Number : 1001

Customer Name : Rahul

Current Balance: 5500.00

(Parameter Safety based verification outputs)

Case 6: Withdraw More Than Balance

Input

Enter your choice : 3

Enter Withdrawal Amount : 7000

Output

Insufficient Balance.

Case 7: Invalid Menu Choice

Input

Enter your choice : 9

Output

Invalid Menu Choice.

Case 8: Negative Initial Balance

Input

Enter your choice : 1

Enter Account Number : 1002

Enter Customer Name : Anjali

Enter Initial Balance : -500

Output

Initial Balance cannot be negative.

Case 9: Negative Deposit Amount**Input**

Enter your choice : 2

Enter Deposit Amount : -1000

Output

Deposit Amount must be positive.

Case 10: Negative Withdrawal Amount**Input**

Enter your choice : 3

Enter Withdrawal Amount : -500

Output

Withdrawal Amount must be positive.

Case 11: Performing an Operation Before Creating an Account**Input**

Enter your choice : 5

Output

Create Account First.

Case 12: Invalid Account Number**Input**

Enter your choice : 1

Enter Account Number : -10

Output

Account Number must be positive.

Case 13: Exit Program**Input**

Enter your choice : 6

Output

Thank You...